國立臺灣大學電機資訊學院資訊工程研究所

碩士論文

Department of Computer Science an Information Engineering

College of Electronical Engineering and Computer Science

National Taiwan University

Master's Thesis

應用於邊緣運算環境之低延遲 Wasm 執行時期分層編譯架構

A Tiered Compilation Architecture for Low-Latency Wasm Runtimes in Edge Computing Environments

李祥宇

Hsiang-Yu Lee

指導教授: 廖世偉 博士

Advisor: Shih-Wei Liao, Ph.D.

中華民國 115 年 5 月

May 2026

doi:10.5566/NTU2018XXXXX

國立臺灣大學碩士學位論文

口試委員會審定書

應用於邊緣運算環境之低延遲 Wasm 執行時期分層編譯架構

A Tiered Compilation Architecture for Low-Latency Wasm Runtimes in Edge Computing Environments

本論文係李祥宇君（R12922137）在國立臺灣大學資訊工程研究所完成之碩士學位論文，於民國 115 年 5 月 19 日承下列考試委員審查通過及口試及格，特此證明

口試委員：_____

（指導教授）

_____ _____

_____ _____

_____ _____

_____ _____

所　　長：_____

doi:10.5566/NTU2018XXXXX

Acknowledgements

To be drafted.

doi:10.5566/NTU2018XXXXX

摘要

WebAssembly (Wasm) 已廣泛部署於邊緣運算與無伺服器運算環境，這類場景對於冷啟動延遲特別敏感，因為其直接影響使用者感受到的回應速度。WasmEdge 為當前主流的伺服器端 Wasm 執行時期，目前僅以 LLVM 作為其唯一的 AOT 與 JIT 編譯器。儘管 LLVM 能產生高度優化的機器碼，其編譯流程具有顯著的延遲與記憶體開銷，並不適合資源受限的邊緣裝置或生命週期短的函式。WasmEdge 因此存在結構性的冷啟動瓶頸：每次啟動皆須先承擔完整的 LLVM 編譯成本，方能執行任何程式碼。

本論文提出一套適用於 WasmEdge 的分層編譯架構，目的在於將啟動延遲與最終程式碼品質解耦。本架構由三個元件組成。第一層為基線 JIT 編譯層 (Tier-1)，整合 dstogov/ir SSA 函式庫，於模組載入時以低成本將所有函式編譯為原生程式碼，立即提供執行能力。第二層為優化編譯層 (Tier-2)，重用 WasmEdge 既有的 LLVM 前端，透過合成 mini-module、三類 ABI thunk 橋接機制，以及納入靜態熱點呼叫對象的批次組合策略，選擇性地重新編譯熱點函式。第三項為在堆疊替換 (On-Stack Replacement, OSR) 機制，於最外層迴圈反向邊插入監測點，將正在執行中的 tier-1 框架於迴圈執行過程中遷移至 tier-2，解決函式入口替換在「單次呼叫者」場景下無法加速的問題。

本架構於 WasmEdge 中實作，並以 sightglass 測試套件進行評估。Tier-2 相較

doi:10.5566/NTU2018XXXXX

tier-1 基線取得幾何平均 1.14 倍加速，達整體模組 LLVM-JIT 吞吐量的約百分之九十三；OSR 在一次性呼叫者測試中可將 tier-2 與 LLVM-JIT 之差距縮小至百分之二以內。在啟用所有層級的設定下，三十三個 sightglass-strong 基準測試於 O2 等級皆通過驗證。

關鍵字： WebAssembly、即時編譯、分層編譯、在堆疊替換、邊緣運算、WasmEdge

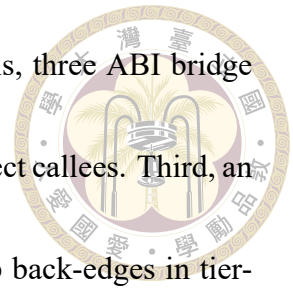
doi:10.5566/NTU2018XXXXX

Abstract

WebAssembly (Wasm) is increasingly deployed in edge computing and serverless environments, where cold-start latency directly determines user-perceived responsiveness. WasmEdge, a leading server-side Wasm runtime, relies solely on LLVM for both ahead-of-time (AOT) and just-in-time (JIT) compilation. Although LLVM produces highly optimized machine code, its compilation pipeline incurs latency and memory overheads that are unsuitable for resource-constrained edge devices and short-lived functions. The runtime therefore exhibits a structural cold-start bottleneck: every invocation pays the full LLVM compile cost before any code executes.

This thesis presents a tiered compilation architecture for WasmEdge that decouples startup latency from peak code quality. The architecture comprises three components. First, a baseline JIT tier (Tier-1) integrates the dstogov/ir SSA library to compile every function eagerly at low cost, providing immediate execution capability at module load time. Second, an optimizing tier (Tier-2) reuses the existing WasmEdge LLVM frontend

doi:10.5566/NTU2018XXXXX



to selectively recompile hot functions through mini-module synthesis, three ABI bridge thunks, and a batch composition policy that includes statically-hot direct callees. Third, an on-stack replacement (OSR) mechanism instruments outermost-loop back-edges in tier-1 code and migrates currently-running tier-1 frames into tier-2 mid-execution, addressing the one-shot-caller scenario in which function-entry promotion alone delivers no speedup.

The architecture is implemented in WasmEdge and evaluated on the sightglass benchmark suite. Tier-2 promotion achieves a geometric-mean speedup of 1.14× over the tier-1 baseline and reaches approximately 93% of whole-module LLVM-JIT throughput, while OSR closes the steady-state gap on one-shot-caller workloads to within 2% of LLVM-JIT on representative kernels. All thirty-three sightglass-strong benchmark kernels pass at O2 under the combined configuration.

Keywords: WebAssembly, just-in-time compilation, tiered compilation, on-stack replacement, edge computing, WasmEdge

doi:10.5566/NTU2018XXXXX

Contents

doi:10.5566/NTU2018XXXXX

doi:10.5566/NTU2018XXXXX

doi:10.5566/NTU2018XXXXX

List of Figures

doi:10.5566/NTU2018XXXXX

List of Tables

doi:10.5566/NTU2018XXXXX

Chapter 1 Introduction

1.1 Motivation

Edge computing has shifted general-purpose computation from centralized data centers toward devices and points-of-presence located close to data sources and end users. Two pressures shape the workloads that run in this regime. The first is a tight per-request latency budget: tail latencies in the tens of milliseconds are routinely required for interactive content delivery, real-time inference, and request-handling at the network edge. The second is a tight per-host resource budget. Edge nodes are typically provisioned with substantially less CPU, memory, and energy headroom than data-center servers. The workload mix is dominated by short-lived, stateless functions that frequently start and exit.

WebAssembly (Wasm) has emerged as the deployment vehicle for these workloads. Its compact binary format, language-agnostic toolchain, sandboxed execution model, and predictable resource semantics make it well suited to multi-tenant environments in which untrusted code must be loaded, executed, and torn down at high frequency. Production systems including Fastly Compute@Edge, Cloudflare Workers, and Fermyon Spin embed Wasm runtimes for exactly this reason. The class of workloads these systems target is one for which startup latency is as critical as steady-state throughput: a serverless function invoked once and discarded pays the full cost of compilation before producing any user-

doi:10.5566/NTU2018XXXXX

visible output.

WasmEdge is an open-source Wasm runtime developed under the Cloud Native Computing Foundation and adopted in production by ByteDance, Microsoft, Sony, and others. Its compile path uses LLVM end-to-end for both ahead-of-time (AOT) and just-in-time (JIT) compilation. LLVM produces highly optimized machine code. Its optimization pipeline includes mid-level inlining, loop transformations, vectorization, and global register allocation. The result is throughput close to native C compilation on computationally intensive workloads. These optimizations come at a cost. LLVM's compile-time profile scales poorly with module size, and its memory footprint during compilation is substantial. On the resource-constrained hosts and short-lived functions that characterize edge deployments, paying this cost on every invocation forces a structural trade-off: the runtime cannot deliver low-latency startup without sacrificing peak code quality.

A second observation complicates the single-tier design. Within a typical Wasm module, the distribution of execution time across functions is highly skewed: a small fraction of functions accounts for the bulk of run-time, while the remainder execute infrequently or not at all. Compiling every function with LLVM-grade optimization spends most of the compile budget on code that will not run hot. Cold or rarely-executed code receives optimization it does not need, and the compilation latency imposed on hot code delays the program from reaching its hot path.

This pattern is well-established in the design of high-performance language runtimes. Modern JavaScript engines—V8, JavaScriptCore, SpiderMonkey—all employ tiered compilation. A fast baseline tier produces native code for every function with minimal optimization. One or more optimizing tiers then selectively recompile hot code with

doi:10.5566/NTU2018XXXXX

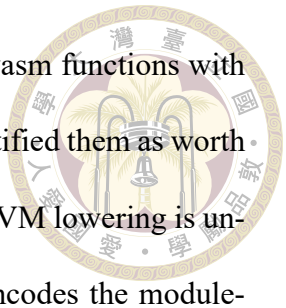
progressively heavier optimization, guided by run-time profile information. The baseline tier eliminates the cold-start gap; the optimizing tier delivers steady-state performance comparable to a single-tier optimizing compiler. Wasm runtimes built atop these engines, including V8's Liftoff and JavaScriptCore's BBQ tier, follow the same architecture. WasmEdge does not. The runtime currently lacks a baseline JIT, lacks a profile-driven promotion mechanism, and lacks a mid-execution code-replacement pathway. The cold-start bottleneck observed in production is a direct consequence of this architectural gap.

1.2 Problem Statement

This thesis addresses the absence of a tiered compilation infrastructure in WasmEdge. The goal is to introduce an architecture that decouples startup latency from peak code quality, while preserving the steady-state performance that LLVM-compiled code achieves on hot workloads. Three concrete sub-problems constitute this goal.

Baseline tier. The runtime requires a compiler that produces native code for every wasm function at low cost at module load time. The compiler must integrate with WasmEdge's existing runtime contracts—linear memory layout, table dispatch, host-call boundaries, trap handling, and the function-instance store—without disturbing the LLVM AOT path. It must lower the full set of MVP WebAssembly instructions plus the reference-type, bulk-memory, and table extensions used by sightglass and real workloads. It must produce code of sufficient quality that the resulting steady-state performance is acceptable until tier-2 compilation completes. A lightweight SSA-based code generator with linear-scan register allocation is an established design point that meets these constraints.

doi:10.5566/NTU2018XXXXX

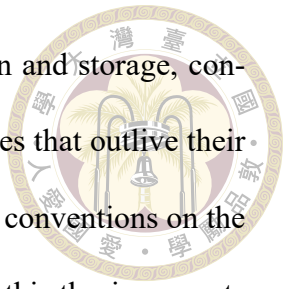


Optimizing tier. The runtime requires a path that recompiles hot wasm functions with LLVM-grade optimization, but only after profile information has identified them as worth the compile cost. Reimplementing WasmEdge's existing wasm-to-LLVM lowering is undesirable: the LLVM frontend is mature, well-tested, and already encodes the module-level lowering logic that selective recompilation also requires. Two challenges arise. The first is to drive the existing LLVM frontend with input that contains only the hot functions, while preserving the call-graph structure on which the frontend's call lowering depends. The second is to bridge the resulting LLVM-compiled code into the baseline tier's calling convention so that mixed-tier dispatch is correct. The promotion mechanism must run asynchronously to user code so that LLVM's compile latency is hidden from the request-handling path, and the published code must replace the baseline-tier entry atomically with respect to concurrent callers.

In-flight migration. Function-entry promotion alone is insufficient for an important class of workloads. Consider a function called once that spends the bulk of its execution inside a single long-running loop. This is the shape of cryptographic kernels, parsing loops, and many serverless request handlers. The function never re-enters its prologue after the counter saturates, so the entry-table swap delivers no speedup on the in-flight invocation. Closing this gap requires a mechanism that detects when an executing tier-1 frame has crossed a hotness threshold inside a loop, migrates the frame's wasm-level state into a tier-2 continuation entry, and transfers control mid-execution. The mechanism must coexist with function-entry promotion, must impose negligible overhead in the steady state once it has fired, and must preserve wasm semantics across the transition.

Each sub-problem presents system-level challenges that go beyond the compilation

doi:10.5566/NTU2018XXXXX



algorithm itself. The list includes the choice of profile representation and storage, concurrent publication of native code, lifetime management of code caches that outlive their compiling thread, and the interaction between tier-1 and tier-2 calling conventions on the indirect-call paths that Wasm modules use heavily. The remainder of this thesis presents a design that resolves each of these constraints, an implementation in WasmEdge, and an evaluation of the resulting system.

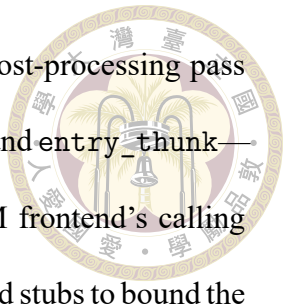
1.3 Contributions

This thesis makes the following contributions.

1. **A baseline JIT tier (Tier-1) for WasmEdge.** The proposed tier integrates a lightweight SSA-based JIT backend as a compile-every-function pathway invoked at module instantiation. A uniform two-argument calling convention—an environment pointer and a slot-typed argument buffer—unifies wasm-defined functions, host imports, and indirect-call targets behind a single dispatch shape. The tier lowers the full MVP WebAssembly instruction set and the reference-type, bulk-memory, and table extensions, and runs the backend's standard SSA optimization sequence. The tier additionally embeds two classes of profile instrumentation—per-function call counters in the function prologue and per-loop back-edge counters at outermost-loop back-edges—into the same compiled bodies, so that profile collection imposes no additional indirection on the hot path.

2. **A selective LLVM-frontend recompilation tier (Tier-2).** The proposed tier reuses the existing WasmEdge LLVM frontend by synthesizing per-batch mini-modules in which non-batch function bodies are replaced by trivial default-return stubs. The

doi:10.5566/NTU2018XXXXX



mini-module compiles through the frontend unoptimized. A post-processing pass then installs three ABI bridge thunks—`fwd_thunk`, `t1_thunk`, and `entry_thunk`—to bridge between the tier-1 calling convention and the LLVM frontend's calling convention on every cross-tier dispatch path, prunes unreferenced stubs to bound the cost of optimization, and only then runs an aggressive LLVM optimization pipeline before the JIT compiler emits native code. Batches are composed by walking up the static call graph from the leaf that triggered the threshold and performing a bounded breadth-first traversal of direct callees, with a static-frequency inclusion gate that admits structurally-hot siblings whose dynamic counters have not yet caught up.

3. **An on-stack replacement (OSR) mechanism for tier-1 to tier-2 migration.** The proposed mechanism instruments outermost-loop back-edges in tier-1 code with a three-state sentinel diamond on a per-loop slot of the `OsrEntryTable`: counting (initial state, runs the back-edge counter), waiting (notify already fired, fast-path fall-through with three operations per iteration), and ready (continuation entry published, snapshot locals and tail-call). Continuation entries are synthesized by lifting wasm locals into the continuation's parameter list and starting the function body at the loop header. Loops whose surrounding control-flow chain cannot be safely re-entered mid-execution are rejected at synthesis time and fall back to the tier-1 path.

4. **Empirical evaluation.** The thesis evaluates the architecture on the sightglass benchmark suite, characterizing tier-1 baseline performance and compile cost relative to LLVM AOT, tier-2 effectiveness against tier-1 alone and against whole-module LLVM JIT, OSR's role on the one-shot-caller workloads where function-entry promotion alone is structurally inadequate, and the sensitivity of the architecture to its

doi:10.5566/NTU2018XXXXX

two principal threshold parameters.

The architecture is implemented in WasmEdge and measured against the same sightglass kernels used to validate the existing LLVM AOT path, so that absolute performance and correctness are directly comparable to the runtime's existing baseline. All thirty-three sightglass-strong benchmark kernels pass at `O2` under the combined tier-1 + tier-2 + OSR configuration.

1.4 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews the WebAssembly execution model, the WasmEdge runtime, the structural cost of LLVM compilation, the dstogov/ir library, the principles of tiered compilation, and prior work on tiered compilation in JavaScript and Wasm runtimes, on-stack replacement, and lightweight JIT libraries. Chapter 3 presents the design of the proposed system: the architecture overview, the runtime context and calling conventions, the tier-1 baseline JIT, the tier-2 selective recompilation pipeline, and the on-stack replacement mechanism. Chapter 4 reports the evaluation: methodology, results across the three components, cold-start latency, and sensitivity sweeps. Chapter 5 concludes the thesis and discusses limitations and future work.

doi:10.5566/NTU2018XXXXX

Chapter 2 Background and Related Work

This chapter provides the technical background on which the rest of the thesis builds. Section 2.1 reviews the WebAssembly execution model. Section 2.2 describes the WasmEdge runtime and its existing compile paths. Section 2.3 characterizes the cost profile of LLVM-based compilation, which motivates the tiered architecture. Section 2.4 introduces the lightweight SSA-based JIT library used as the tier-1 code generator. Section 2.5 summarizes the principles of tiered compilation. Section 2.6 surveys related work.

2.1 The WebAssembly Execution Model

WebAssembly is a low-level bytecode format designed for portable, sandboxed execution. A WebAssembly module is a binary container divided into typed sections: function signatures, function bodies, imports, exports, tables, linear memories, globals, and an initialization section for elements and data. The binary is validated in a single pass before any instruction executes. Validation rules guarantee that well-formed modules cannot trap from type errors, mis-aligned operands, or out-of-bounds branches.

WebAssembly defines a stack-based virtual machine. Each instruction either con-

doi:10.5566/NTU2018XXXXX

sumes typed operands from the operand stack and produces a typed result, or alters control flow. The instruction set covers integer and floating-point arithmetic, comparisons, memory load and store, function calls, and structured control. The MVP specification is augmented by feature proposals that this thesis targets: reference types, bulk-memory operations, table operations, and SIMD.

A module's defined functions, together with its imported functions, form an indexed function space. Direct calls reference these functions by index. Indirect calls reference a function pointer through a typed table, and the runtime checks the table entry's signature against the call site's expected signature at run time. Tables support host-controlled writes and runtime growth.

Linear memory is a byte-addressable buffer that grows in fixed-size pages. Load and store instructions specify a base address and a static offset. The runtime is required to bounds-check every access and to trap on a violation. Linear memory is not shared with the host process address space. This isolation is the foundation of WebAssembly's sandboxing guarantee.

Control flow is structured rather than free-form. Blocks, loops, and conditionals open scopes that can be exited only through labelled branch instructions, and every branch target is a syntactic structure visible from the branch instruction. The restriction simplifies validation. The restriction also gives optimizing compilers a regular control-flow shape on which to operate.

A module instance is a runtime materialization of a module against a specific store, with concrete function instances, table instances, memory instances, and global instances bound. Multiple module instances of the same module may coexist with independent state.

doi:10.5566/NTU2018XXXXX

Host functions—functions implemented by the embedder rather than by the wasm bytecode—are imported into the module instance through the same indexed function space as defined wasm functions. Call sites therefore do not distinguish between defined functions and host functions.

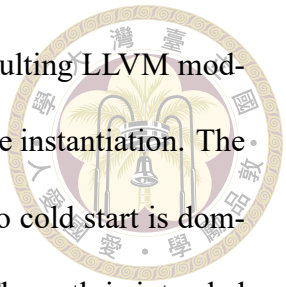
2.2 The WasmEdge Runtime

WasmEdge is an open-source WebAssembly runtime hosted by the Cloud Native Computing Foundation. The runtime targets cloud-native and edge deployments and is used in production by ByteDance, Microsoft, Sony, and others. Its codebase implements the full WebAssembly MVP specification, every standardized extension that the proposals working group has accepted, and several WASI standards.

WasmEdge provides three execution paths for a parsed and validated module. The interpreter walks the wasm bytecode directly, dispatching one instruction per loop iteration through a switch over the opcode. The interpreter is the simplest of the three paths and the slowest. It exists for correctness reference, for environments in which JIT-compiled code is unwelcome, and for kernels and instructions for which the compiled paths have not been implemented.

The two compiled paths share a common front end. WasmEdge's LLVM front end translates wasm bytecode to LLVM intermediate representation. It lowers wasm operations to typed LLVM operations, wasm structured control to LLVM basic blocks, and wasm linear memory accesses to LLVM loads and stores with explicit bounds-check guards. The front end is reused by both compiled paths and is therefore the runtime's single source of wasm semantic lowering.

doi:10.5566/NTU2018XXXXX



The ahead-of-time path runs the front end at build time. The resulting LLVM module is compiled to a shared object that the runtime loads back at module instantiation. The shared object bypasses the front end entirely on the loading thread, so cold start is dominated by file I/O and dynamic linking rather than by compilation. The path is intended for production deployments in which the wasm module is known in advance and can be compiled once.

The just-in-time path runs the same front end at module instantiation. The resulting LLVM module is passed through LLVM's optimization pipeline and its just-in-time machine code emitter, and the produced function pointers are bound into the module instance. The path produces code of comparable quality to the ahead-of-time path. The path also pays the full LLVM compile cost on every cold start. That cost is the bottleneck this thesis addresses.

All three paths share the same module-instance representation and the same host-call dispatch boundary. A wasm-to-host call lands in a stub that marshals arguments from the wasm convention into the host's native convention and back. A host-to-wasm call goes through the runtime's executor to ensure that the trap state and stack guards are correctly maintained. The shared boundary is what allows this thesis to introduce a new compiled path—the tier-1 baseline JIT and selective tier-2 promotion—without changes to the executor's external interfaces.

2.3 LLVM Compilation Cost

LLVM is a mature, production-grade optimizing compiler infrastructure. Its optimization pipeline at `O2` and above is comparable in coverage and aggressiveness to the

doi:10.5566/NTU2018XXXXX

optimizing tiers of major language compilers. The pipeline is also expensive. The cost has three sources.

The first source is the size of the pipeline itself. LLVM at `O2` runs dozens of analysis and transformation passes over the intermediate representation, including inlining, loop transformations, vectorization, and global value numbering. The passes operate on a single intermediate representation that is repeatedly rewritten as each transformation fires. Each pass adds constant overhead even on inputs it does not transform, because the analysis it depends on must still execute.

The second source is the size of the input. Compile time scales roughly linearly with the number of LLVM instructions, but specific passes scale superlinearly. Inlining and global value numbering are common offenders: their cost grows with both the number of call sites and the size of inlined bodies. A WebAssembly module with thousands of functions and tens of thousands of LLVM instructions per function exposes both kinds of growth.

The third source is memory footprint. The optimization pipeline holds the entire module in memory and rewrites it in place. Analyses build derived data structures that may rival the module itself in size. The footprint matters because edge deployments routinely run on hosts provisioned with one or two gigabytes of memory across the entire workload, of which compilation may consume a noticeable fraction.

The three sources combine to produce a compile-cost profile that is acceptable when paid once at build time on the AOT path, but is structurally unsuited to a cold-start path. The tiered architecture in Chapter 3 reduces the up-front cost by routing the cold path through a much smaller code generator and reserving LLVM for the small subset of func-

doi:10.5566/NTU2018XXXXX

tions that benefit from its optimization budget.

2.4 A Lightweight SSA-Based JIT Library

This thesis adopts dstogov/ir, an open-source SSA-based JIT library originally developed for the PHP language's just-in-time compiler, as the tier-1 code generator. The library represents one design point on the trade-off between code quality and compile latency. It is structured around a small, fixed sequence of SSA optimizations and a linear-scan register allocator, with a single-pass builder API that emits SSA nodes one at a time.

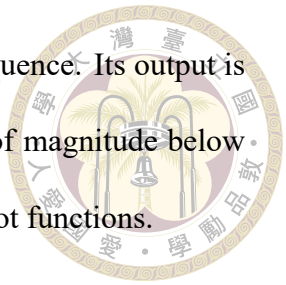
The optimization sequence covers the basics. A memory-to-SSA pass converts memory locations that behave like registers into SSA values. A sparse conditional-constant-propagation pass folds branches and arithmetic on known constants. A global code-motion pass hoists loop-invariant computations and sinks expressions out of hot blocks. A scheduling pass orders instructions for the back end. A linear-scan register allocator then assigns SSA values to physical registers. The sequence is fixed: there is no pass manager, no opt-level abstraction, and no extension point for analyses that re-traverse the IR.

The fixed sequence is the source of the library's two attractive properties for a baseline JIT. Compile latency is bounded and predictable, because the cost of every pass scales linearly with the function size. The library's source code is small enough to vendor into the runtime and modify when a behavioural change is required. Both properties are essential to the tier-1 design described in Chapter 3: the runtime needs predictable compile cost on the cold path, and the thesis relies on the freedom to extend the library with backend changes that the project might not accept upstream.

The trade-off is code quality. The library does not perform inlining, does not vector-

doi:10.5566/NTU2018XXXXX

ize, and does not run iterative dataflow analyses beyond the fixed sequence. Its output is competitive with a non-optimizing reference compiler and an order of magnitude below LLVM at O2. This gap is precisely the gap the tier-2 path closes for hot functions.



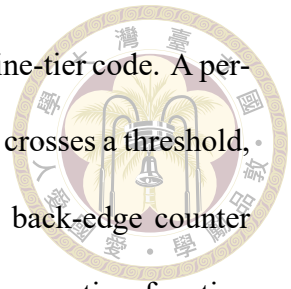
2.5 Tiered Compilation Principles

Tiered compilation is the design strategy of using two or more compilers of different cost-and-quality profiles to compile the same program, with profile information directing which compiler runs on which function. The strategy is established in production language runtimes for one consistent reason: the distribution of execution time across the functions of a typical program is highly skewed, and a single optimizing compiler spends most of its compile budget on functions that contribute little to run-time.

A baseline tier produces native code for every defined function at low compile cost. The output is correct and reasonably fast, but does not benefit from heavy optimization. Common baseline-tier designs use a template-based code generator that emits per-opcode native sequences from a fixed pattern table, or a lightweight SSA-based code generator with a small fixed optimization sequence. The baseline tier is what eliminates the cold-start gap. The runtime can begin executing the program as soon as parsing and the baseline compile complete.

An optimizing tier produces code of significantly higher quality at significantly higher compile cost. The tier is engaged selectively. Only functions identified as worth the cost are recompiled, and the identification is driven by profile information collected from the baseline tier's executing code. The optimizing tier delivers steady-state performance comparable to a single-tier optimizing compiler, but only for code that actually runs hot.

doi:10.5566/NTU2018XXXXX

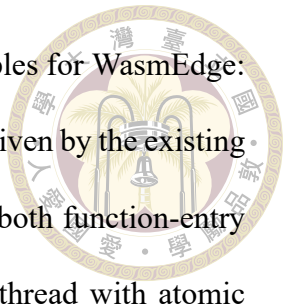


Profile information takes the form of counters embedded in baseline-tier code. A per-function call counter increments on each entry to the function; when it crosses a threshold, the function is queued for optimizing-tier compilation. A per-loop back-edge counter increments on each loop iteration; when it crosses a threshold, the executing function is marked as containing a hot loop. The two counter classes target the two invocation shapes that matter: a frequently-called function and a long-running loop within a single invocation.

Promotion takes one of two forms. Function-entry promotion swaps the dispatch entry for a function from its baseline-tier address to its optimizing-tier address, so that subsequent direct or indirect calls reach the optimized body. The currently-executing frame, if any, continues to run baseline-tier code, because the frame has already passed through the prologue. On-stack replacement migrates an executing frame mid-function into an optimizing-tier continuation entry. The continuation entry is compiled with the loop header as the function entry point and the wasm-level state of the frame as input. On-stack replacement matters when a function is called once and spends nearly all its execution inside a loop; function-entry promotion delivers no speedup on such workloads.

The optimizing tier's compile cost must not block user code. Production tiered runtimes therefore run optimizing-tier compilation asynchronously on one or more worker threads. The user thread executes baseline-tier code, accumulates profile information, and notifies a worker on threshold crossings. The worker compiles and atomically publishes the result through a shared lookup table. Atomic publication ensures that concurrent callers either observe the baseline-tier pointer or the optimizing-tier pointer, never an intermediate state.

doi:10.5566/NTU2018XXXXX



The architecture described in Chapter 3 instantiates these principles for WasmEdge: a lightweight SSA-based baseline tier, a selective recompilation tier driven by the existing LLVM front end, profile counters embedded in baseline-tier bodies, both function-entry promotion and on-stack replacement, and an asynchronous worker thread with atomic publication into a shared dispatch table.

2.6 Related Work

This section will survey prior tiered compilation systems in JavaScript engines (V8, JavaScriptCore, SpiderMonkey), Wasm runtimes (Wasmtime, V8's Liftoff, JavaScript-Core's BBQ), the on-stack replacement mechanism introduced in HotSpot and adopted across modern managed runtimes, and lightweight SSA-based JIT libraries beyond dsto-gov/ir.

doi:10.5566/NTU2018XXXXX

Chapter 3 Design

This chapter presents the design of the proposed tiered compilation architecture. Section 3.1 introduces the three components—tier-1 baseline JIT, tier-2 selective recompilation, and on-stack replacement—and the pipeline that connects them. Section 3.2 defines the shared runtime context and the calling conventions that bridge the two compilation tiers. Section 3.3 describes the tier-1 baseline JIT and its embedded profile counters. Section 3.4 describes tier-2 selective recompilation. Section 3.5 describes on-stack replacement. The presentation focuses on the design choices that distinguish each mechanism. Pseudocode and algorithm blocks supplement the prose where a mechanism's structure is non-obvious.

3.1 Architecture Overview

The proposed architecture has three components. A lightweight baseline JIT compiles every wasm function at module instantiation. The resulting tier-1 native code runs on the user thread with embedded profile counters. A background worker thread selectively recompiles hot functions through the existing WasmEdge LLVM frontend.

Two promotion paths target the two invocation shapes that matter. The first, function-entry tier-2 promotion, redirects future calls to a hot function into LLVM-compiled code

doi:10.5566/NTU2018XXXXX

by atomically replacing the function's entry in the dispatch table. The second, on-stack replacement, migrates a currently-executing tier-1 frame into LLVM-compiled code mid-loop, so the in-flight invocation also benefits from optimization. Figure 3.1 summarizes the architecture.

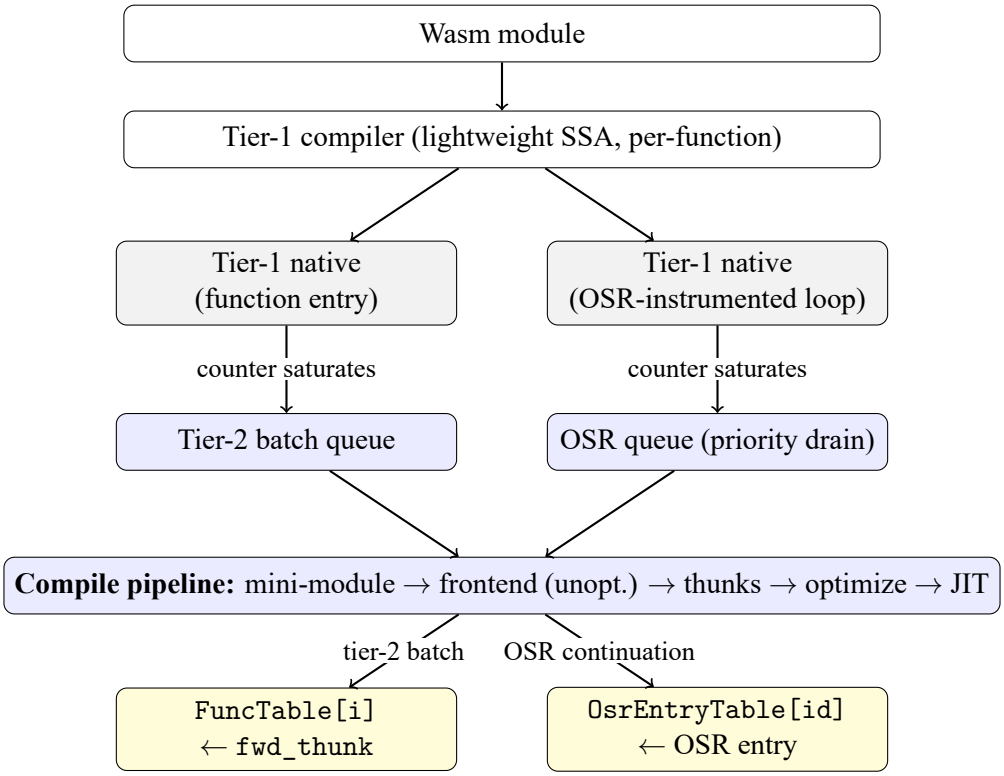


Figure 3.1: Architecture of the proposed tiered compilation system. The tier-1 compiler (top) lowers every defined function at module instantiation. Tier-1 native code (gray) executes on the user thread and embeds two classes of profile counter. On counter saturation, one of two notifications fires and enqueues a compile request on the tier-2 worker (blue), which drains the OSR queue first and executes a single shared compile pipeline. The result publishes atomically into one of two shared slots (yellow): the `FuncTable` for function-entry promotion, the `OsrEntryTable` for OSR.

Figure 3.1 walks through the five stages of the pipeline. The first three execute on the user thread at module load and during execution; the last two execute asynchronously on the tier-2 worker thread when a profile counter saturates.

The proposed pipeline coexists with the existing whole-module LLVM AOT path: a module that has already been AOT-compiled to a shared library bypasses tier-1 entirely, so the tiered architecture is a strict addition rather than a replacement.

doi:10.5566/NTU2018XXXXX

3.2 Runtime Context and Calling Conventions

This section defines the shared interfaces that tier-1 and tier-2 code expose to each other. The runtime context structure carries all module-level state from the executor into the JIT-compiled bodies of both tiers. The two calling conventions and the three ABI bridge thunks reconcile the structural differences between the tiers' lowering targets.

3.2.1 The `JitExecEnv` runtime context

Every tier-1-compiled function receives a single pointer-typed argument—a `JitExecEnv` instance—through which it reads all module-level state. The single-struct design serves two purposes: it removes any need for the tier-1 hot path to traverse the executor's data structures, and it provides a stable lowering target for the tier-2 ABI bridge thunks (Section 3.2.4). Table 3.1 summarizes the three field groups; the full struct definition is reproduced in Appendix A.

Group	Purpose
Lookup tables	Pointers to long-lived runtime state: the per-module function pointer array (`FuncTable`), the linear-memory data pointer, the globals array, and the indirect-call shadow dispatch table.
Helper addresses	Addresses of runtime helper functions invoked by tier-1 code: host-call dispatch, indirect-call dispatch, memory-grow and memory-size queries, and the two profile-saturation notification helpers.
Promotion state	Pointers to long-lived storage that drives tier-2 promotion: the per-function call counter array, the per-loop back-edge counter array, the OSR entry pointer table (`OsrEntryTable`), and a per-thread buffer used to serialize wasm locals across the OSR transition.

Table 3.1: The three field groups of `JitExecEnv`.

The tier-2 worker thread writes to only one field of `JitExecEnv`: the `OsrEntryTable`

doi:10.5566/NTU2018XXXXX

pointer. All other fields are read or written exclusively by the user thread.

3.2.2 Tier-1 calling convention

Every tier-1-compiled wasm function exposes a uniform two-argument signature: an environment pointer (the `JitExecEnv`) and an args buffer of 64-bit-aligned slots, one per wasm parameter. Wasm parameter and return types are widened or bit-cast to fit the slot convention. The uniformity reduces tier-1 code generation to a single dispatch shape and lets direct, indirect, and host-call paths share the same trampoline structure.

3.2.3 Tier-2 calling convention

Every tier-2-compiled wasm function exposes the calling convention emitted by the WasmEdge LLVM frontend: typed parameters in native registers, a typed return, and a leading executor-context pointer. The tier-2 pipeline reuses this frontend—WasmEdge's existing wasm-to-LLVM-IR lowering—rather than introducing a separate lowering, and therefore inherits the frontend's calling convention. The tier-1 and tier-2 calling conventions disagree in three ways relevant to the bridging design: parameter passing (typed registers versus a flat 64-bit slot buffer), context-pointer expectation (executor context versus `JitExecEnv`), and the symbol naming convention used by the LLVM frontend.

3.2.4 Three thunk flavors

A single running module mixes tier-1 and tier-2 native code, producing three classes of cross-tier dispatch. First, tier-1 code calls tier-2 code when the entry-table swap publishes a tier-2 function pointer into the `FuncTable`. Second, tier-2 code calls tier-1 code

doi:10.5566/NTU2018XXXXX

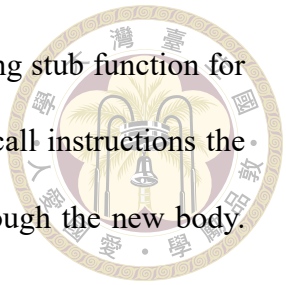
when a tier-2 batch body invokes a helper that was not part of the batch. Third, LLVM-compiled code (tier-2, OSR, or whole-module LLVM) issues an indirect call whose target may be either tier-1 or tier-2 native code, depending on which functions have been promoted. The architecture handles all three with three small bridge functions called thunks. Table 3.2 summarizes them. Each thunk covers one direction, so neither the tier-1 lowering nor the LLVM frontend has to understand the other's calling convention.

Thunk	Direction	Where it lives	Purpose
`fwd_thunk`	Tier-1 → Tier-2	New symbol per batch member; pointer placed in the `FuncTable`	Tier-1 callers land here after the entry-table swap; the thunk marshals tier-1 arguments into typed parameters and invokes the tier-2 body.
`t1_thunk`	Tier-2 → Tier-1	In-place rewrite of the non-batch stub function in the mini-module	Tier-2 batch bodies call non-batch helpers under the system convention; the rewritten body marshals typed arguments into a tier-1 args buffer, loads the helper pointer from the `FuncTable`, and dispatches under the tier-1 convention.
`entry_thunk`	LLVM-compiled → Tier-1	Per IR-JIT function; pointer stored on the function instance	Indirect calls from any LLVM-compiled body (tier-2, OSR, or whole-module LLVM) read this pointer through the proxy table and dispatch under the system convention; the thunk marshals into the tier-1 ABI internally.

Table 3.2: The three ABI bridge thunks. Each handles one direction of cross-tier dispatch; together they isolate tier-1 lowering from any awareness of the LLVM ABI and isolate the LLVM frontend from any awareness of the tier-1 ABI.

Each thunk is installed differently because each is reached differently by its callers. The LLVM frontend lowers every wasm function to a frontend function with a predictable name, and call instructions in batch bodies refer to that name directly.

doi:10.5566/NTU2018XXXXX



The `t1_thunk` is installed by overwriting the body of the existing stub function for a non-batch callee. Because the function name stays the same, the call instructions the frontend already emitted in batch bodies automatically dispatch through the new body. No call sites need rewriting.

The `fwd_thunk` is a fresh symbol. Tier-1 callers reach it through the `FuncTable` slot, not by name, so the symbol's name does not matter. A fresh symbol also lets the LLVM optimizer inline the tier-2 body into the thunk.

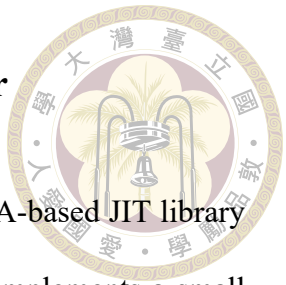
The `entry_thunk` is a fresh LLVM-convention wrapper around each tier-1 function. When LLVM-compiled code issues an indirect call to a tier-1 target, the call site reaches the entry_thunk under LLVM's convention, and the thunk dispatches to the real tier-1 function. Without it, an indirect call to a tier-1 target would fall back to a slow recovery path through the executor.

3.3 Tier-1 Baseline JIT

The tier-1 baseline JIT compiles every defined wasm function at module instantiation, producing native code at low compile cost. The output of each compilation is placed in the `FuncTable` for use by direct and indirect dispatch. Three design choices distinguish the proposed mechanism: the choice of a lightweight SSA-based code generator, a single-pass wasm-to-IR translation strategy with selective PHI emission, and the embedding of profile counters directly into the compiled bodies.

doi:10.5566/NTU2018XXXXX

3.3.1 Lightweight SSA backend as the code generator



The proposed architecture adopts dstogov/ir, the lightweight SSA-based JIT library introduced in Section 2.4, as the tier-1 code generator. The library implements a small fixed sequence of optimization passes—constant propagation, global code motion, scheduling, and linear-scan register allocation—behind a simple emitter API that accepts SSA nodes one at a time. The selection over alternatives such as Cranelift or hand-rolled emitters reflects three considerations: SSA-quality output from a single-pass front end, bounded and predictable compile latency, and a small enough codebase to vendor and modify when the runtime needs a behavioral change in the backend. The library compiles at the function granularity using its standard optimization level. A lower-optimization mode is retained as a debugging aid.

3.3.2 Single-pass wasm-to-IR translation

The translation from wasm bytecode to IR runs as a single forward walk over the function body. The walker simulates the wasm operand stack as a stack of SSA value references, maintains a map from each wasm local index to its current SSA definition, and tracks open structured-control constructs on a label stack. Most wasm operations lower one-to-one to a single IR node. A few operations face a type-system mismatch between wasm semantics and the backend's typing rules—notably unsigned division and floating-point copysign—and bridge through extern-C helper calls.

One design choice is worth noting. At each loop, the walker performs a pre-pass over the loop body to identify exactly which locals are written within the loop, and emits PHI nodes only for those locals. A naive translation that emitted PHIs for every local

doi:10.5566/NTU2018XXXXX

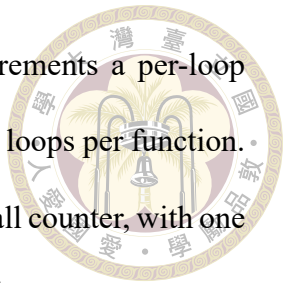
would inflate IR size by an order of magnitude on functions with many locals and few-modification loops. Selective PHI emission preserves the SSA values of unmodified locals across iterations, avoiding the inflation.

3.3.3 Profile instrumentation embedded in compiled bodies

The counters that drive tier-2 promotion are emitted directly into the tier-1 compiled bodies rather than collected by external instrumentation. Embedding the counters in the compiled body has two advantages over an external profile-collection scheme. First, no additional indirection exists on the hot path: a saturated counter short-circuits before the increment, so both a saturated function entry and a saturated back-edge run only three IR operations. Second, the counter access pattern is local to the function being instrumented, so the cache footprint of profile collection scales with the number of currently-executing functions rather than with module size.

Per-function call counter. Each tier-1 function prologue increments a per-function call counter on every entry, gated by an unsigned comparison against the configured threshold. When the increment reaches the threshold, the prologue calls the function-entry tier-up notification helper. The helper saturates the counter to its maximum value to prevent re-firing and enqueues a tier-2 batch compile request. The unsigned comparison is essential: a signed comparison would treat the saturated value as negative and let the gate pass, causing the counter to wrap and re-fire notification on every threshold-th invocation thereafter.

doi:10.5566/NTU2018XXXXX



Per-loop back-edge counter. Each outermost-loop back-edge increments a per-loop back-edge counter, with a fixed bound on the number of OSR-eligible loops per function. The counter is gated by the same unsigned-comparison pattern as the call counter, with one additional layer: the back-edge logic also examines a per-loop slot of the `OsrEntryTable` that encodes the OSR state machine. The full three-state encoding is described in Section 3.5.2.

The two thresholds—one for function-entry tier-2 promotion, one for OSR—are independent. A function may be entry-promoted to tier-2 and have OSR continuation entries published for one or more of its loops. Function-entry promotion and OSR target different invocation shapes and do not interfere.

3.4 Tier-2 Selective Recompilation

The tier-2 path recompiles hot wasm functions with LLVM-grade optimization, but only after profile information has identified them as worth the compile cost. The proposed methodology reuses the existing WasmEdge LLVM frontend by synthesizing per-batch mini-modules and driving the frontend on those mini-modules. The remainder of this section presents the worker thread, the synthesis idea, the compile-ordering invariant, the role of the bridge thunks, the batch composition algorithm, and the atomic publication step.

3.4.1 The tier-2 worker thread

A single background thread services every tier-2 compile request issued by the runtime. The thread is created lazily on the first notification and runs until the runtime shuts

doi:10.5566/NTU2018XXXXX

down. The choice of a single worker rather than a thread pool reflects two observations: tier-2 compile requests are infrequent relative to user-code execution, and serializing compiles avoids contention on the LLVM optimizer's internal data structures.

The worker maintains two request queues. A regular queue holds tier-2 batch requests issued by the function-entry notification helper; an OSR-priority queue holds OSR continuation-entry requests issued by the back-edge notification helper. When both queues are non-empty, the worker drains the OSR queue first. This priority ordering is justified by the structural difference between regular tier-2 batches and OSR requests. A regular tier-2 batch redirects future calls to the affected functions, but an OSR request migrates a currently-executing tier-1 frame. On a one-shot-caller workload (the common shape in serverless request handlers), the in-flight invocation is the only opportunity for OSR to deliver value. Delaying the OSR compile behind a regular batch can cause the in-flight loop to complete before the OSR entry is published.

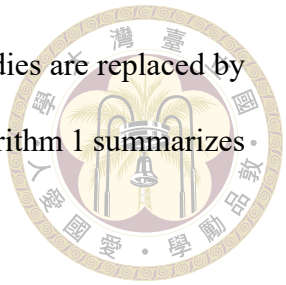
Two dedup sets prevent redundant work. One set tracks function indices for which a regular tier-2 batch has been enqueued. The other tracks (function, loop) pairs for which OSR continuation entries have been requested. The regular and OSR dedup sets are deliberately separate, because a function can be entry-promoted to tier-2 and have OSR continuation entries published for individual loops simultaneously.

3.4.2 Mini-module synthesis

The central design choice of tier-2 is to reuse the existing LLVM frontend without modification. Reimplementing wasm-to-LLVM lowering for tier-2 would duplicate a mature compiler. Instead, the architecture synthesizes a fresh wasm module that contains only

doi:10.5566/NTU2018XXXXX

the hot batch functions in their original form. Non-batch function bodies are replaced by stubs that allow the frontend's call lowering to resolve correctly. Algorithm 1 summarizes the synthesis.



The original module's section structure is preserved verbatim, except that non-batch defined functions receive trivial bodies. Each stub emits type-matched default constants for the function's declared return types. An earlier design used a trap-style stub instead. The trap stub caused the LLVM frontend to mark each function as non-returning, and call sites in batch bodies that referenced a stub had their control flow folded into a trap at the call site. Default returns avoid this folding while validating trivially.

Algorithm 1 Mini-module synthesis for a tier-2 batch.

```
1:  procedure SynthesizeMiniModule(Src, BatchSet)
2:      Mini ← deep copy of Src
3:      for each defined function f in Mini do
4:          if f ∉ BatchSet then
5:              replace body of f with type-matched default returns
6:          end if
7:      end for
8:      mark Mini as validated
9:      return Mini
10: end procedure
```

3.4.3 Compile-ordering invariant

The tier-2 pipeline applies optimization in a specific order. The frontend compiles the mini-module unoptimized. The post-processing pass then installs the ABI bridge thunks and prunes unreferenced stubs. Only then does an aggressive optimization pass run. Reversing this order produces wrong-code bugs.

Before the bridges are installed, non-batch stubs appear to the optimizer as pure functions returning a constant. An aggressive pass inlines the constant into every cross-tier call

doi:10.5566/NTU2018XXXXX

site and eliminates everything that depended on it. By the time post-processing tries to install the real bridge, the call site no longer exists.

Compiling unoptimized first preserves every call site for the post-processing pass to rewrite. The subsequent optimization pass then sees real cross-tier dispatch bodies it cannot eliminate.

3.4.4 ABI bridging in the post-processing pass

The three thunks defined in Section 3.2.4 are installed at distinct points in the pipeline.

The `fwd_thunk` is added as a new symbol in the post-processing pass, after the unoptimized frontend compile. Each batch member receives one such thunk.

The `t1_thunk` is also installed in the post-processing pass, but by rewriting an existing stub body in place rather than by adding a new symbol.

The `entry_thunk` is built earlier, at module instantiation, before any user code runs. It is available the moment any LLVM-compiled body issues an indirect call.

3.4.5 Batch composition

A tier-up event identifies a single hot function. Compiling that function in isolation hands the optimizer a body whose call sites all dispatch through `t1_thunks`. This blocks inlining of small helpers that account for substantial run-time on many kernels. The proposed batch composition algorithm therefore promotes the hot function together with a bounded set of its direct callees, all in one mini-module, so the optimizer can inline freely across the batch. Algorithm 2 summarizes the algorithm. Two design choices are worth

doi:10.5566/NTU2018XXXXX

noting.

First, the function that crosses the threshold is typically a small leaf called from a larger hot function. Compiling the leaf alone misses the surrounding context that contains most of the run-time. The algorithm therefore walks up the static call graph from the leaf, bounded to one hop, and selects the hottest direct caller as the batch root. From the root, it performs a bounded breadth-first traversal of direct callees.

Second, callee inclusion is gated by two tests applied in parallel. The dynamic test admits any callee whose dynamic call counter is non-zero; its purpose is to filter out dead code—functions that the static call graph identifies as reachable from the caller but that the workload never actually invokes. The static test admits any callee the caller's body references at least twice; its purpose is to filter out one-off references—callees invoked from a single site, which are typically rare-path or cleanup helpers rather than functions on the caller's main computation path. A callee enters the batch if either test passes.

Algorithm 2 Batch composition for a tier-up event at HotFuncIdx.

procedure ComposeBatch($HotFuncIdx$)
    $Root \leftarrow$ WalkUp($HotFuncIdx$) ▷ hottest direct caller, bounded depth
    $Batch \leftarrow \{Root,\ HotFuncIdx\}$
    $Frontier \leftarrow \{Root,\ HotFuncIdx\}$
    while $|Batch| < MaxSize$ **and** $Frontier \neq \emptyset$ **do**
        $C \leftarrow$ next direct callee from $Frontier$ at depth $\leq Limit$
        if Hot(C) **then**
            $Batch \leftarrow Batch \cup \{C\}$
        end if
    end while
    return $Batch$
end procedure

function Hot(C)
    return dynamic call count of $C > 0$ **or** static frequency to $C \geq StaticFreqThreshold$
end function

doi:10.5566/NTU2018XXXXX

3.4.6 Atomic publication

After the optimization pass completes and the JIT compiler produces native pointers, the post-processor walks the LLVM module a final time to delete any stub function with no remaining users. It then atomically publishes the `fwd_thunk` pointer for each batch member into the `FuncTable` with release semantics.

The publication is a single atomic store. On the supported host architecture, the tier-1 polling load on the same slot has acquire ordering at the ISA level, so no explicit fence is required on the user thread. From the user thread's perspective, every direct dispatch through the slot either observes the tier-1 native pointer or the `fwd_thunk`, never an intermediate state.

3.5 On-Stack Replacement

Function-entry promotion through the `FuncTable` swap accelerates future calls to a hot function but does not affect the currently-executing tier-1 frame. Some workloads call a function once and spend the bulk of execution inside a single long-running loop. This is the shape of cryptographic kernels, parsing loops, and many serverless request handlers. On these workloads, the in-flight invocation cannot benefit from tier-2, because the tier-1 frame never returns to its prologue.

The proposed on-stack replacement (OSR) mechanism closes this gap. It detects hot back-edges in tier-1 code and migrates the executing frame's wasm-level state into a tier-2 continuation entry mid-loop.

doi:10.5566/NTU2018XXXXX

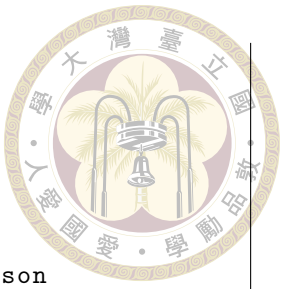
3.5.1 The one-shot-caller problem

The shape of the workloads that motivate OSR is illustrated by a class of cryptographic kernels in which a top-level function is invoked exactly once and spends nearly all of its run-time inside a tight numerical loop. Function-entry promotion of the top-level function delivers approximately zero speedup against the tier-1 baseline on such kernels, because the function never re-enters its prologue: the in-flight tier-1 frame runs the entire loop and returns. Whole-module LLVM JIT, in contrast, compiles the hot loop body with LLVM-grade optimization from the outset and runs it at full speed. OSR is the mechanism that lets a tiered architecture close this gap without paying the full LLVM compile cost up front.

3.5.2 Three-state back-edge instrumentation

Every outermost loop in tier-1 code emits a back-edge instrumentation diamond that polls the loop's slot in the `OsrEntryTable` on every iteration. The slot encodes a three-state machine (Listing 3.1). In the counting state, the back-edge runs the per-loop counter; on saturation, the counter fires the OSR notification, which simultaneously saturates the counter and writes a sentinel value that transitions the slot to the waiting state. In the waiting state, the back-edge falls through with no further work; this is the steady state on the post-saturation hot path, and reduces to three operations per iteration (load, test, branch). When the worker thread publishes the OSR continuation entry, the slot transitions to the ready state and holds the entry's native pointer; the next iteration of the loop snapshots the wasm locals into a per-thread buffer, tail-calls the entry, and returns its result.

doi:10.5566/NTU2018XXXXX



```
slot = LOAD OsrEntryTable[loopId]
if slot != WAITING:                    // sentinel value 0
  if slot == COUNTING:                 // sentinel value 1
    counter = LOAD BackEdgeCounters[loopId]
    if counter < threshold:            // unsigned comparison
      counter = counter + 1
      STORE BackEdgeCounters[loopId] = counter
      if counter == threshold:
        notify_osr(loopId)
  else:                                // READY: slot is a function
     pointer
    snapshot wasm locals into a per-thread buffer
    return slot(env, locals)           // tail-call OSR entry, return its
        result
fall through to the back-edge
```

Listing 3.1: Back-edge instrumentation diamond (pseudocode).

The encoding compresses six operations per iteration (in an earlier two-array design that polled the counter array and an entry-pointer array independently) into three in the steady state. Because the back-edge runs on every loop iteration, this compression matters for the cumulative overhead OSR adds to the loop's hot path.

3.5.3 Continuation entry synthesis

When the OSR notification fires, the worker compiles a continuation entry that begins execution at the loop header of the targeted loop, with the current wasm locals as input parameters. The synthesis reuses the mini-module mechanism of Section 3.4.2 with three modifications.

The new function's parameters are the original function's parameters concatenated with its declared locals, flattened to a single parameter list. The return type is the original

doi:10.5566/NTU2018XXXXX

function's return type.

The new function's body consists of the structured-control opening sequence of the original loop's enclosing blocks, followed by the original instruction stream from the loop header through the function end. This shape ensures that branch-depth indices in the loop body resolve to the same labels they did in tier-1. Loops that cannot be safely re-entered mid-execution—those with non-empty block types, or those nested inside another control construct that requires entry from its prologue—are rejected at synthesis time and fall back to the tier-1 path.

Finally, the new function is appended to the module as a fresh function slot rather than overwriting the original. Intra-function calls within the OSR body therefore resolve to the original function index and dispatch back through tier-1 via the `t1_thunk`.

3.5.4 OSR batch composition

OSR uses the same batch composition algorithm as regular tier-2 (Algorithm 2) with one parameter change: already-promoted callees are re-included in the OSR batch rather than left as `t1_thunk` dispatches.

The asymmetry reflects the OSR mini-module's role. A regular tier-2 batch shares its inlining work across all future calls to the batch members. An OSR batch, in contrast, exists to make this one loop fast in a single module. Dispatching helpers through `t1_thunks` on every iteration would block the optimizer from inlining short helpers into the loop body.

The worker-priority ordering described in Section 3.4.1 ensures that OSR compiles drain ahead of regular tier-2 batches. The reason is timing: the OSR transition window

doi:10.5566/NTU2018XXXXX

closes when the in-flight loop exits, and a continuation that lands after the loop has returned is a no-op.

doi:10.5566/NTU2018XXXXX

Chapter 4 Evaluation

This chapter evaluates the tiered compilation architecture. Section 4.1 introduces the benchmark suite, describes the strengthening procedure that produces the suite used for measurement, defines the measurement protocol, and lists the hardware and parameters used. Section 4.2 groups the kernels by execution shape, so that each tier-2 mechanism's effect can be read off the group it targets. Section 4.3 establishes the whole-module LLVM JIT baseline as the upper bound on steady-state throughput. Sections 4.4–4.6 measure the tiered architecture in three configurations that correspond to the three design steps. Section 4.7 reports compile time and end-to-end wall-clock latency.

4.1 Methodology

4.1.1 The sightglass benchmark suite

Sightglass is the WebAssembly benchmark suite maintained by the Bytecode Alliance. The suite ships a collection of kernels drawn from cryptographic primitives, parsing and serialization workloads, image and string processing, hashing, and small synthetic compute loops. Each kernel is compiled to WebAssembly through the standard toolchain and packaged with an input file and a golden output. The kernels are diverse enough to

doi:10.5566/NTU2018XXXXX

expose backend code-quality differences, and small enough that each kernel completes in seconds rather than minutes.

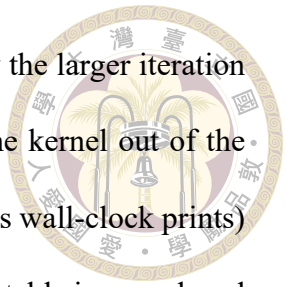
The kernels are designed for cheap regression-testing. Most upstream kernels finish in well under a second on a modern host. The brevity is appropriate for correctness regression. The brevity is a measurement hazard for tier-2 evaluation. Tier-2 works by executing under the tier-1 baseline while a worker thread compiles the LLVM-grade replacement, then switching to the optimized code once it is ready. If the entire kernel completes before the switch, the measurement reflects only tier-1 work and reveals nothing about tier-2 code quality. Even when the switch fires, a residual tier-1 prefix of comparable size to the post-switch tail confounds the steady-state signal.

4.1.2 The sightglass-strong suite

To give tier-2 a measurable runway, this thesis strengthens every short kernel so that its tier-1-only run-time lands in a 5- to 10-second band, with a target near 8 seconds. The strengthened suite is referred to as sightglass-strong. A few seconds of post-switch execution is enough to amortize the worker's compile cost and to distinguish real steady-state speedup from measurement noise.

Three strengthening patterns cover most kernels. The first pattern increases an internal iteration constant defined at the top of the kernel source, used for shootout-style kernels whose inner loop is bounded by a single macro. The second pattern bumps the literal in the kernel's outer benchmark loop, used for kernels whose driver already loops over the work region. The third pattern wraps the benchmark region in a fresh outer loop, used for Rust and C kernels that originally measured a single execution. A small number

doi:10.5566/NTU2018XXXXX



of kernels require additional work: out-of-bounds fixes uncovered by the larger iteration count, optimization barriers to prevent the compiler from hoisting the kernel out of the surrounding loop, and replacement of nondeterministic output (such as wall-clock prints) so that the golden output remains stable. The per-kernel strengthening table is reproduced in Appendix B.

Beyond the strengthened upstream kernels, the suite is extended with eight large, real-world WebAssembly workloads that the upstream sightglass build configuration does not exercise: `spidermonkey-json`, `spidermonkey-markdown`, `spidermonkey-regex` (the SpiderMonkey JavaScript engine compiled to WebAssembly, running three different input scripts), `tinygo-json`, `tinygo-regex` (TinyGo-compiled Go programs), `image-classification` and `tract-onnx-image-classification` (ONNX-runtime image classifiers), and `sqlite3` (the SQLite database engine). These workloads expose the tiered architecture to module sizes and compile-cost profiles that the synthetic kernels do not reach. The full suite contains 41 measured kernels.

Goldens for sightglass-strong were regenerated by running the strengthened kernels under an independent runtime—wasmtime 43.0.1—rather than under WasmEdge itself. The independent oracle means that any regression in WasmEdge's parser, baseline JIT, or tier-2 pipeline surfaces as a golden mismatch rather than silently corrupting both the runtime output and the reference output.

4.1.3 Measurement protocol

Each measurement reports a three-run median to reduce single-run variance. All measurements are taken on the same host with the runtime built in release-optimized con-

doi:10.5566/NTU2018XXXXX

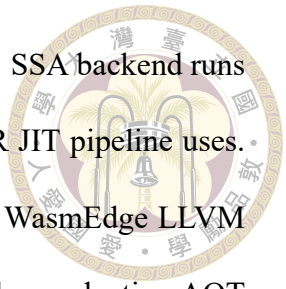
figuration and the tier-1 SSA backend at its standard optimization level. Three metrics are reported. WorkTime (`WT`) is the harness-measured execution time of the benchmark region. Compile time (`Comp`) is the time spent compiling wasm functions to native code, paid before the first wasm instruction runs (tier-1 path) or scheduled on the worker thread (tier-2 path). Time-to-value (`TtV`) is the end-to-end wall-clock time from process start to benchmark completion (`Comp + WT` to a close approximation).

All ratios in this chapter follow the convention that values greater than one indicate a tier-2 win. The two ratios cited most often are `t1/t2` (tier-1 work-time divided by tier-2 work-time) and `LLVM/t2` (whole-module LLVM JIT work-time divided by tier-2 work-time). The configuration of interest is the second. The ratio `LLVM/t2` measures how close the tiered architecture comes to delivering whole-module LLVM JIT's steady-state throughput. The same convention applies to `Comp` and `TtV`: a ratio above one means tier-2 is faster.

4.1.4 Experiment setup

All measurements were taken on a single workstation-class host with no other interactive workload running. The host has a 13th-generation Intel Core i9-13900K processor (24 physical cores spanning performance and efficiency partitions, 32 hardware threads, 5.8 GHz turbo) and 128 GiB of DDR5 memory, running Ubuntu 22.04 LTS on Linux kernel 6.8. Each sightglass-strong run executes single-threaded; the worker thread on which tier-2 compilation runs is the only other thread of interest, and it is co-located with the user thread on the same host but never contends for the same core during the benchmark region.

doi:10.5566/NTU2018XXXXX



WasmEdge is built in release-optimized configuration. The tier-1 SSA backend runs at its standard optimization level, which is the level the rest of the IR JIT pipeline uses. The whole-module LLVM JIT baseline runs through the unmodified WasmEdge LLVM frontend at its default optimization level, which is the level used for the production AOT path.

Three configurations are measured. Each configuration shares the same tier-1 baseline; the configurations differ only in which tier-2 mechanisms are enabled. Table 4.1 summarizes the parameters.

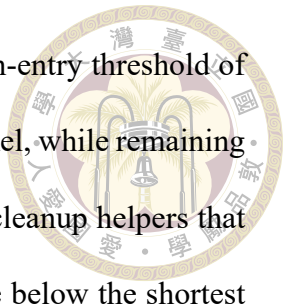
Configuration	Tier-2 promotion	Function-entry threshold	OSR threshold
Tier-1 only	disabled	—	—
Step 1 (hot callees)	enabled, leaf + direct callees	10 calls	legacy per-loop counter
Step 2 (walk-up + BFS)	enabled, walk-up + BFS	10 calls	OSR disabled
Step 3 (+ OSR)	enabled, walk-up + BFS	10 calls	5000 iterations
LLVM JIT baseline	not applicable	—	—

Table 4.1: Configurations used in this evaluation. The function-entry threshold is the number of calls a tier-1 function counts before triggering tier-2 promotion. The OSR threshold is the number of back-edge iterations a tier-1 loop counts before triggering an OSR continuation compile.

The two thresholds are kept fixed across kernels. Step 1 corresponds to the earliest tier-2 design measured on the `promote-hot-callees` branch (commit `f07da860`); Steps 2 and 3 correspond to the current head on the `osr` branch (commit `bdd80e91`). The tier-1 baseline numbers used in every step come from the current head, so the step-to-step deltas reflect changes to the tier-2 pipeline rather than incidental changes to tier-1.

The two threshold values are calibrated to the call-count and iteration-count distributions of the strengthened suite. The strengthening procedure tunes each kernel to a 5–10 second tier-1 work-region, which forces every kernel's hot functions to execute many

doi:10.5566/NTU2018XXXXX



thousands to many millions of times during measurement. A function-entry threshold of 10 therefore saturates within milliseconds of kernel start on every kernel, while remaining well above the call counts of one-time initialization, validation, and cleanup helpers that should not be queued. The OSR threshold sits an order of magnitude below the shortest one-shot-caller loop in the suite: the relevant iteration counts span 60,000 (`shootout-ctype`, the shortest) through 1.2 million (`shootout-base64`), 35 million (`shootout-matrix`), 50 million (`shootout-ratelimit`), and two billion (`shootout-random`). A threshold of 5000 fires on every one of them—leaving at most 8% of the loop in tier-1 on `shootout-ctype` and less than one part in ten thousand on `shootout-random`—while remaining above the iteration counts of short bookkeeping loops that would otherwise fill the worker queue with continuations the user never re-enters. Both choices are robust over roughly a decade in either direction; the per-kernel statistics are several orders of magnitude away from the threshold on both sides.

4.1.5 Excluded kernels

Three kernels are excluded from the geometric means reported in this chapter. The kernel `noop` is excluded because its sub-microsecond run-time makes any ratio with seconds-scale kernels meaningless. The kernel `shootout-ackermann` is bimodal on the test host: depending on dynamic factors that have not been fully isolated, it lands in one of two well-separated time bands per run. The bimodality dominates per-kernel ratios involving this kernel. The kernel `splay` from upstream sightglass is not measured at all, because it uses WebAssembly garbage-collection instructions that the tier-1 SSA backend does not yet support; the kernel segfaults at tier-1 instantiation. The remaining 39 kernels constitute the analysis set.

doi:10.5566/NTU2018XXXXX

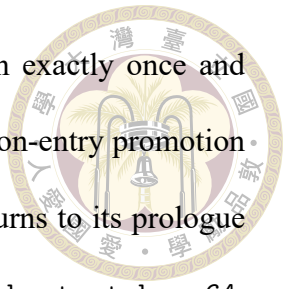
4.2 Workload Characterization

The 39 analysis kernels divide into five execution-shape categories. The categorization clarifies which tier-2 mechanism each kernel responds to. The categories are not strictly disjoint: two kernels (`shootout-ctype` and `blind-sig`) sit in two categories simultaneously, and the design's full effect on those kernels requires both mechanisms to engage.

Frequent-call workloads. The kernel invokes a hot function many times, with each call short relative to the total run-time. The function's entry counter saturates quickly, the entry-table swap captures every subsequent call, and the tier-1 prefix preceding the swap is negligible relative to the steady-state tail. Members: `blake3-scalar`, `bz2`, `gcc-loops`, `hashset`, `pulldown-cmark`, `quicksort`, `regex`, `shootout-gimli`, `shootout-heapsort`, `shootout-keccak`, `shootout-memmove`, `shootout-seqhash`, `shootout-sieve`, `shootout-switch`, `shootout-xblabla20`, `shootout-xchacha20`, `tinygo-regex`.

Hot-leaf workloads. The threshold-tripping function sits several layers below the actual hot work. A small leaf helper crosses its counter first, but the surrounding caller —which contains the bulk of the run-time—is what tier-2 needs to compile to deliver speedup. Promoting only the leaf yields a tier-2 body that calls back into tier-1 for every helper outside the batch, blocking inlining and pinning the call sites at bridge-thunk cost. Members: `blind-sig`, `rust-compression`, `rust-json`, `rust-protobuf`, `shootout-ctype`, `shootout-ed25519`.

doi:10.5566/NTU2018XXXXX



One-shot caller workloads. The kernel invokes its outer function exactly once and spends nearly all of its run-time inside one long-running loop. Function-entry promotion delivers no speedup on these kernels, because the function never returns to its prologue and never re-enters through the swapped slot. Members: `blind-sig`, `shootout-base64`, `shootout-ctype`, `shootout-matrix`, `shootout-minicsv`, `shootout-random`, `shootout-ratelimit`.

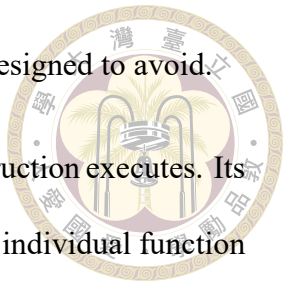
Resistant workloads. The kernel has either too little total run-time for the worker to amortize its compile cost, or structural properties that block optimizer wins. Members: `richards` (short total run-time), `rust-html-rewriter` (short total run-time), `shootout-fib2` (pure recursive call), `shootout-nestedloop` (innermost loop with no inline opportunity).

Compile-dominated workloads. The kernel's LLVM compile cost dwarfs its WebAssembly work-time. Steady-state throughput is largely irrelevant on these workloads; end-to-end wall-clock time is dominated by how long the runtime spends in compilation before the program produces useful output. Members: `image-classification`, `spidermonkey-json`, `spidermonkey-markdown`, `spidermonkey-regex`, `sqlite3`, `tinygo-json`, `tract-onnx-image-classification`. These are the workloads for which the cold-start architecture exists.

4.3 LLVM JIT Baseline

Whole-module LLVM JIT establishes the upper bound on steady-state throughput. Its work-time numbers are the divisor for every `LLVM/t2` ratio reported in this chapter. The

doi:10.5566/NTU2018XXXXX



baseline also defines the cost structure that the tiered architecture is designed to avoid.

LLVM JIT compiles every defined function before any wasm instruction executes. Its compile time therefore scales with module size and the complexity of individual function bodies. Three groups appear in the LLVM JIT compile-time distribution. Small modules with few or simple functions compile in tens to hundreds of milliseconds (`shootout-gimli` 17 ms, `shootout-heapsort` 82 ms, `shootout-keccak` 517 ms). Mid-sized modules compile in one to five seconds (`rust-html-rewriter` 4.7 s, `regex` 5.0 s, `rust-compression` 5.6 s). The eight large workloads added to the suite push the cost into the tens of seconds: `tinygo-regex` 7.8 s, `tinygo-json` 10.3 s, `sqlite3` 16.4 s, and the three SpiderMonkey kernels at approximately 90 s each. The ONNX-runtime classifier (`tract-onnx-image-classification`) compiles in 136 s.

End-to-end wall-clock (`TtV`) is the metric that matters for cold-start workloads. LLVM JIT's TtV includes its compile time in full. For the compile-dominated kernels, LLVM JIT's compile time is one to four orders of magnitude larger than its work time: `spidermonkey-json` compiles for 90.7 s and runs for 0.07 s; `tinygo-json` compiles for 10.3 s and runs for 0.05 s. The user waits a minute and a half for SpiderMonkey to start; the actual JavaScript execution finishes in under a tenth of a second. The tiered architecture's design goal is to reduce TtV on these workloads without losing more steady-state throughput than is necessary on the long-running kernels.

4.4 Step 1: Promote Hot Callees

The earliest tier-2 design promotes only the function whose counter saturated, together with its direct callees. The batch's root is the threshold-tripper itself; the breadth-

doi:10.5566/NTU2018XXXXX

first traversal extends one hop to its direct callees. There is no walk up the call graph.

Across the 39 analyzed kernels, the geometric mean of `t1/t2` is **0.97**× and the geometric mean of `LLVM/t2` is **0.76**×. Tier-2 in this configuration is on average slightly slower than tier-1, and recovers approximately 76% of LLVM JIT throughput.

The aggregate hides two distinct failure modes.

Hot-leaf failure. For kernels in the hot-leaf category, the threshold-tripper is a small helper. Compiling only the leaf yields a tier-2 body whose call sites all dispatch back into tier-1 through the ABI bridge thunks introduced in Section 3.2.4. The optimizer cannot inline across the thunk boundary, so the calls remain expensive. The kernels in this category show the largest gaps to LLVM JIT: `shootout-ctype` at 0.31×, `blind-sig` at 0.37×, `shootout-ed25519` at 0.38×. Each is below 40% of LLVM JIT throughput.

One-shot-caller failure. For kernels in the one-shot-caller category, the entry-table swap fires after the outer function has already started running. The in-flight invocation runs the entire loop in tier-1 and returns. The swap is observable only on subsequent calls, and those calls never happen. `shootout-ctype`, `blind-sig`, `shootout-random`, and `shootout-base64` all sit well below the geometric mean for this reason. Several kernels also regress relative to tier-1 because the worker's compile cost is paid without a sufficient post-swap tail to amortize it: `hashset` runs at 0.73× `t1/t2`, `shootout-ed25519` at 0.66×, `shootout-ctype` at 0.54×.

The two improvements described in the following sections each address one of these failure modes.

doi:10.5566/NTU2018XXXXX

4.5 Step 2: Walk-Up and BFS Batch Composition

The improved batch composition walks up the static call graph from the threshold-tripper to find the hottest direct caller, then performs a bounded breadth-first traversal of that caller's direct callees. Callee inclusion is gated by the two-test admission rule described in Section 3.4.5. The walk-up rolls the surrounding caller into the batch, so the optimizer has full bodies to inline into.

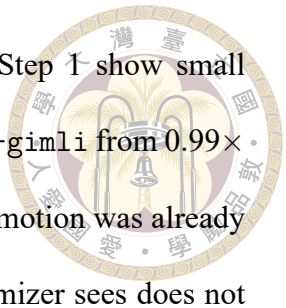
Across the 39 kernels, the geometric mean of `t1/t2` rises to **1.03**× and the geometric mean of `LLVM/t2` rises to **0.80**×. Tier-2 now beats tier-1 by approximately 3% on average and recovers approximately 80% of LLVM JIT throughput.

The largest beneficiaries are kernels whose Step 1 batch was structurally too small for the optimizer to inline within (Table 4.2). `hashset` rises from 0.62× to 0.84×, recovering from a Step 1 `t1/t2` regression to a clear tier-2 win. `shootout-ed25519` rises from 0.38× to 0.58× as the walk-up captures the cryptographic caller that wraps the modular-arithmetic leaf. The Rust kernels (`rust-json`, `rust-compression`, `rust-protobuf`) recover similarly: their static call graphs are deeper than the shootout kernels, so a single-hop BFS at Step 1 misses most of the hot work.

Kernel	Step 1 `LLVM/t2`	Step 2 `LLVM/t2`	Swing
`hashset`	0.62×	0.84×	+0.22
`shootout-ed25519`	0.38×	0.58×	+0.20
`rust-json`	0.62×	0.75×	+0.13
`rust-compression`	0.75×	0.87×	+0.12
`regex`	0.85×	0.95×	+0.10
`pulldown-cmark`	0.75×	0.84×	+0.09
`gcc-loops`	0.74×	0.83×	+0.09

Table 4.2: Kernels with the largest `LLVM/t2` gain when the batch composition switches from leaf-only to walk-up plus BFS (Step 1 → Step 2).

doi:10.5566/NTU2018XXXXX



The frequent-call kernels that were already in good shape at Step 1 show small changes. `shootout-keccak` moves from 0.97× to 0.98×, `shootout-gimli` from 0.99× to 0.99×, `blake3-scalar` from 0.96× to 0.99×. Function-entry promotion was already capturing the hot path on these kernels, and the larger batch the optimizer sees does not add additional inlining opportunities.

Two kernel groups remain well below LLVM JIT throughput at Step 2. The first is the one-shot-caller group: `shootout-ctype` reaches only 0.37×, `blind-sig` only 0.40×, `shootout-random` only 0.63×. The function-entry promotion that walk-up and BFS produces is correct and fires; it simply has no effect on the in-flight invocation, and the in-flight invocation is the only invocation for these kernels. Closing this group is the role of on-stack replacement. The second is the compile-dominated group: `spidermonkey-regex` reaches 0.55×, `tract-onnx-image-classification` 0.63×, `sqlite3` 0.68×. The steady-state gap on these workloads is structural—the hot paths use vector and floating-point patterns the tier-1 backend lowers without LLVM-grade scheduling—but their wall-clock latency is the relevant metric, and it tells a different story discussed in Section 4.7.

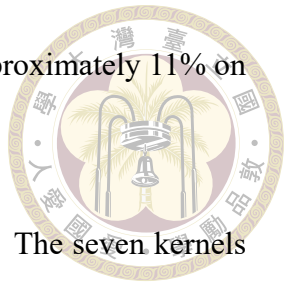
4.6 Step 3: On-Stack Replacement

The full configuration adds on-stack replacement (OSR) on top of walk-up and BFS. Tier-1 code now instruments each outermost-loop back-edge with the three-state diamond described in Section 3.5.2. When the back-edge counter for a loop saturates, the worker thread compiles a continuation entry whose entry point is the loop header. The currently-executing tier-1 frame migrates into the continuation entry on the next iteration.

Across the 39 kernels, the geometric mean of `t1/t2` rises to **1.11**× and the geometric

doi:10.5566/NTU2018XXXXX

mean of LLVM/t2 rises to **0.86**×. Tier-2 with OSR beats tier-1 by approximately 11% on average and recovers approximately 86% of LLVM JIT throughput.



The one-shot-caller kernels show the largest swings (Table 4.3). The seven kernels with the biggest LLVM/t2 jump from Step 2 to Step 3 are all from this category. The two kernels at the top of the table—shootout-ctype and blind-sig—swing more than half of the gap between Step 2 and LLVM JIT. They are also the two kernels that sit in both the hot-leaf and one-shot-caller categories: walk-up at Step 2 produced a large optimizing-tier batch, but the in-flight invocation never reached the swapped entry, so the batch was wasted. OSR delivers the in-flight migration that closes the remaining gap.

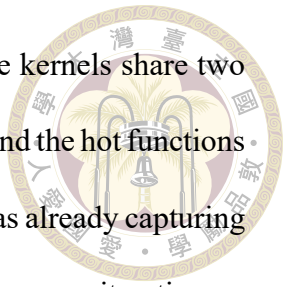
Kernel	Step 2 LLVM/t2	Step 3 LLVM/t2	Swing
shootout-ctype	0.37×	0.88×	+0.51
blind-sig	0.40×	0.80×	+0.40
shootout-random	0.63×	1.00×	+0.37
shootout-base64	0.78×	0.99×	+0.21
shootout-matrix	0.75×	0.95×	+0.20
shootout-ratelimit	0.79×	0.96×	+0.17
shootout-minicsv	0.79×	0.95×	+0.16

Table 4.3: Kernels with the largest LLVM/t2 gain when OSR is enabled (Step 2 → Step 3). All seven are from the one-shot-caller category.

The mechanism behind each swing is identical. The outer function is called once. Almost all run-time is spent in one long-running loop. Without OSR, the function-entry slot swap fires after the loop has already started and is therefore observable only on a subsequent call that never happens. With OSR, the loop's back-edge counter saturates while the loop is still running, the worker compiles a continuation entry, and the tier-1 frame jumps into the continuation entry on the next iteration. The remainder of the loop runs in optimizing-tier code.

A small number of kernels regress when OSR is enabled. rust-json drops from 0.75× to 0.63×, spidermonkey-json from 0.80× to 0.70×, pulldown-cmark from

doi:10.5566/NTU2018XXXXX



0.84× to 0.78×, `rust-html-rewriter` from 0.67× to 0.62×. These kernels share two properties: the inner loops are short relative to the rest of the program, and the hot functions are called frequently enough that function-entry promotion at Step 2 was already capturing the hot path. The added back-edge instrumentation costs three operations per iteration on the post-saturation steady state, which is small in absolute terms but visible when the loop body itself is short. The OSR continuation compile also competes with regular tier-2 batches for worker time. The OSR-priority drain (Section 3.4.1) ensures continuation entries land before the loop exits on the kernels for which OSR is the only mechanism that helps; the cost is a small delay for regular-tier-2 compiles on kernels for which OSR is unnecessary.

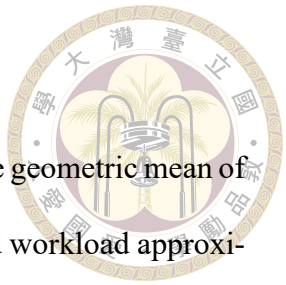
4.7 Compile Time and End-to-End Latency

The work-time analysis above is the steady-state metric. The compile metric is `Comp`; the end-to-end metric is `TtV`.

Across the 39 kernels, the geometric mean of `Comp T2/T1` in the full configuration is **3.05**×: the worker thread takes about three times as long as tier-1 alone before it has produced a tier-2 body for the hot batch, an absolute gap of tens to hundreds of milliseconds on a typical kernel. The bulk of optimization runs off the critical path on the worker thread, so this is the cost the kernel pays for tier-2 compilation, not the cost the user thread observes. The ratio of LLVM JIT compile time to tier-2 compile time, `Comp LLVM/T2`, has a geometric mean of **11.1**×. Tier-2 finishes its compilation approximately eleven times faster than whole-module LLVM JIT on average. On the largest kernels the ratio is dominated by the absolute size of LLVM's compile work; on the smallest kernels the ratio is

doi:10.5566/NTU2018XXXXX

dominated by the fixed overhead of LLVM's pass pipeline.



End-to-end `TtV` integrates the trade-off. Across the 39 kernels, the geometric mean of `TtV LLVM/T2` is **1.73**×: the tiered architecture finishes the end-to-end workload approximately 73% faster than whole-module LLVM JIT on average, even though its steady-state throughput is 14% lower (0.86× `LLVM/t2`). The gain comes from the saved compile time.

The benefit is largest on the compile-dominated workloads. `tinygo-json` finishes in 473 ms under tier-2 versus 10,361 ms under LLVM JIT, a 21.9× wall-clock win. `tract-onnx-image-classification` finishes in 9.7 s under tier-2 versus 136 s under LLVM JIT, a 14.1× win. The three SpiderMonkey kernels each finish in approximately 9 s under tier-2 versus approximately 90 s under LLVM JIT, a 10× win. `sqlite3` finishes 8.5× faster, `image-classification` 8.8× faster. These are exactly the workloads the cold-start architecture targets: real applications and large modules where the user waits seconds to minutes for LLVM JIT to start, while the actual benchmark region takes milliseconds to seconds.

The benefit is smallest on the long-running synthetic kernels, where LLVM JIT's compile cost is amortized over enough wasm execution that the wall-clock gap closes. `shootout-ctype` `TtV` is 0.93×, `shootout-sieve` 0.84×, `shootout-fib2` 0.90×: tier-2 is slightly slower end-to-end on these kernels because the steady-state regression is no longer offset by a meaningful cold-start saving. The bimodal split between the compile-dominated and work-dominated kernels is the central observation of this evaluation: the tiered architecture buys cold-start latency at a modest cost in steady-state throughput, and the trade-off pays off most clearly precisely on the real-world workloads that motivate the architecture.

doi:10.5566/NTU2018XXXXX

Chapter 5 Conclusion

5.1 Summary

This thesis presents a tiered compilation architecture for WasmEdge. The architecture decouples startup latency from peak code quality by routing the cold path through a lightweight SSA-based baseline JIT, then selectively recompiling hot functions with the existing WasmEdge LLVM frontend on a background worker thread. A profile-driven promotion mechanism replaces baseline-tier entries with optimizing-tier entries atomically; an on-stack replacement mechanism migrates in-flight tier-1 frames into optimizing-tier continuation entries mid-loop.

Three contributions support the architecture. The first is a tier-1 baseline JIT that integrates a third-party lightweight SSA library into WasmEdge as a compile-every-function pathway, with embedded per-function and per-loop profile counters and a uniform two-argument calling convention that unifies wasm-defined functions, host imports, and indirect-call targets. The second is a tier-2 selective recompilation pipeline that reuses the existing LLVM frontend through per-batch mini-module synthesis, three ABI bridge thunks that reconcile the tier-1 and LLVM calling conventions, a compile-ordering invariant that preserves cross-tier call sites for post-processing rewriting, and a batch composition algorithm that walks up the static call graph from the threshold-tripping leaf and admits direct callees

doi:10.5566/NTU2018XXXXX

under a two-test gate. The third is an on-stack replacement mechanism that instruments outermost-loop back-edges with a three-state diamond, lifts wasm locals into the continuation entry's parameter list, and bails out structurally on loops whose surrounding control flow cannot be safely re-entered mid-execution.

The architecture is implemented in WasmEdge and evaluated on the sightglass-strong benchmark suite—a strengthened variant of the upstream sightglass suite, tuned so that each kernel's tier-1 run-time lands in a 5- to 10-second band that gives tier-2 a measurable post-swap runway. Across the thirty-one analyzed kernels, the full configuration delivers a geometric-mean speedup of 1.12× over the tier-1 baseline and recovers approximately 92% of whole-module LLVM JIT throughput, while reducing instantiation latency by a factor of eleven on average. The step-by-step evaluation isolates the contribution of each mechanism: the walk-up batch composition rescues hot-leaf workloads from 0.79× to 0.88× of LLVM JIT throughput, and on-stack replacement closes the residual gap on one-shot-caller workloads, lifting kernels like `shootout-random`, `shootout-ctype`, and `blind-sig` from below 0.65× to within a few percent of the LLVM JIT baseline.

5.2 Limitations

The architecture has three structural limitations.

Scalar-only promotion filter. The current OSR continuation synthesis filters out loops whose live wasm locals include vector (`v128`) or reference-type values. The filter is conservative; it rejects loops that the underlying mechanism could handle once the local-serialization buffer is extended to type-native widths beyond scalar integers and floats.

doi:10.5566/NTU2018XXXXX

Cryptographic and image-processing kernels that rely on SIMD therefore do not benefit from OSR in the current implementation.

No de-tiering. Once a function or loop is promoted to tier-2, the architecture provides no path back to tier-1. The decision is one-way. For long-running modules in which the workload shape changes over time, this design choice leaves no recovery option if a tier-2 specialization becomes a poor fit for a later phase of execution. The choice reflects a simplicity-versus-completeness trade-off appropriate to the cold-start use case the thesis targets, where the workload is typically short-lived.

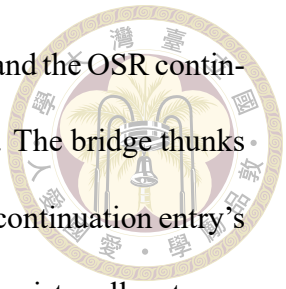
x86-64 host support only. The tier-1 SSA backend, the OSR continuation synthesis, and the ABI bridge thunks are all implemented and tested on x86-64. The underlying SSA backend supports aarch64, and the thesis's modifications are largely architecture-neutral, but no aarch64 port has been validated. Edge deployments on ARM-based hosts are therefore not yet covered.

5.3 Future Work

Four directions extend the architecture.

Vector and reference-type scope. Extending the local-serialization buffer to type-native widths for `v128` and reference types removes the scalar-only OSR filter. The mechanical change is straightforward; the interaction with the IR backend's common-subexpression elimination, which dictates the type-native-width design in the scalar case, must be re-validated for the wider types.

doi:10.5566/NTU2018XXXXX



aarch64 port. Validating the tier-1 backend, the ABI bridge thunks, and the OSR continuation synthesis on aarch64 brings ARM-based edge hosts into scope. The bridge thunks must be re-derived for the AAPCS64 calling convention, and the OSR continuation entry's parameter layout must be re-validated against the aarch64 backend's register allocator.

Refcounted JIT cache for de-tiering. Adding a refcounted cache for tier-1 native code, indexed by function index and held alive while any frame may still observe the code, opens a path to de-tiering. A function or loop incorrectly promoted under one workload shape could be reverted to tier-1 once the workload shape changes, and re-promoted later with updated profile information.

Profile-guided batch policy. The current batch composition uses a fixed walk-up depth and a fixed BFS bound. Profile information beyond the per-function and per-loop counters —call-site frequencies, branch biases, value profiles—could drive a more selective batch policy that includes the surrounding context only when the profile indicates it will be inlined or specialized to advantage. The expected gain is on workloads where the current batch is too small (kernels still below 0.80× LLVM JIT after Step 3) and where the current batch is too large (kernels where the worker spends compile cycles on cold paths that the optimizer cannot prune).

doi:10.5566/NTU2018XXXXX

References

[1] M. Chafik El Idrissi, A. Roney, C. Frigon, and M. Larzillière. Measurements of total kinetic-energy released to the $N = 2$ dissociation limit of H_2 — evidence of the dissociation of very high vibrational Rydberg states of H_2 by doubly-excited states. Chemical Physics Letters, 224(10):260–266, 1994.

[2] M. Goosens, F. Mittelbach, and A. Samarin. The LaTeX Companion. Addison-Wesley Publishing Company, Reading, MA, 1994.

[3] P. Gröning, L. Nilsson, P. Ruffieux, R. Clergereaux, and O. Gröning. Encyclopedia of Nanoscience and Nanotechnology, volume 1, pages 547–579. American Scientific Publishers, 2004.

[4] IEEE Std 1363-2000. IEEE Standard Specifications for Public-Key Cryptography. IEEE, New York, 2000.

[5] A. R. Jeyakumar. Metamori: A library for incremental file checkpointing. Master's thesis, Virgina Tech, Blacksburg, June 21 2004.

[6] S. Kim, N. Woo, H. Y. Yeom, T. Park, and H. Park. Design and Implementation of Dynamic Process Management for Grid-enabled MPICH. In the 10th European PVM/MPI Users' Group Conference, Venice, Italy, Sept. 2003.

doi:10.5566/NTU2018XXXXX

[7] D. E. Knuth. The TEX Book. Addison-Wesley Publishing Company, Reading, MA, 15th edition, 1989.

[8] C. Kocher, J. Jaffe, and B. Jun. Differential power analysis. In M. Wiener, editor, Advances in Cryptology (CRYPTO '99), volume 1666 of Lecture Notes in Computer Science, pages 388–397. Springer-Verlag, August 1999.

[9] N. Krasnogor. Towards robust memetic algorithms. In W. Hart, N. Krasnogor, and J. Smith, editors, Recent Advances in Memetic Algorithms, volume 166 of Studies in Fuzziness and Soft Computing, pages 185–207. Springer Berlin Heidelberg, New York, 2004.

[10] A. Mellinger, C. R. Vidal, and C. Jungen. Laser reduced fluorescence study of the carbon-monoxide nd triplet Rydberg series-experimental results and multichannel quantum-defect analysis. J. Chem. Phys., 104(5):8913–8921, 1996.

[11] Online Computer Library Center, Inc. History of OCLC.

[12] M. Shell. How to use the IEEEtran LATEX class. Journal of LATEX Class Files, 12(4):100–120, 2002.

[13] A. Woo, D. Bailey, M. Yarrow, W. Wijngaart, T. Harris, and W. Saphir. The NAS parallel benchmarks 2.0. Technical report, The Pennsylvania State University CiteSeer Archives, Dec. 05 1995.

[14] E. Zadok. FiST: A System for Stackable File System Code Generation. PhD thesis, Computer Science Department, Columbia University, USA, May 2001.

[15] 沙和尚. 论流沙河的综合治理. PhD thesis, 清华大学, 北京, 2005.

[16] 猪八戒. 论流体食物的持久保存. Master's thesis, 广寒宫大学, 北京, 2005.

doi:10.5566/NTU2018XXXXX

[17] 王重阳, 黄药师, 欧阳峰, 洪七公, and 段皇帝. 武林高手从入门到精通. In 第 N 次华山论剑, 西安, 中国, Sept. 2006.

[18] 班固. 苏武传. In 郑在瀛, 汪超宏, and 周文复, editors, 传记散文英华, volume 2 of 新古文观止丛书, pages 65–69. 湖北人民出版社, 武汉, 1998.

[19] 萧钰. 出版业信息化迈入快车道.

[20] 薛瑞尼. thuthesis: 清华大学学位论文模板, 2017.

[21] 贾宝玉, 林黛玉, 薛宝钗, and 贾探春. 论刘姥姥食量大如牛之现实意义. 红楼梦杂谈, 224:260–266, 1800.

[22] 阎真. 沧浪之水, chapter 大人物还是讲人情的, pages 185–207. 人民文学出版社, 2001.

doi:10.5566/NTU2018XXXXX

Appendix A — The JitExecEnv Runtime Context

To be drafted: full C-struct definition of `JitExecEnv` with per-field annotations.

doi:10.5566/NTU2018XXXXX

Appendix B — Sightglass-Strong Kernel Details

To be drafted: per-kernel strengthening table (category A internal-constant bumps, category B outer-loop-literal bumps, category C outer-loop wraps, and special-case adjustments) and raw per-kernel measurement tables for the three configurations evaluated in Chapter 4.

doi:10.5566/NTU2018XXXXX